

WEEK-3

Session-11: Date and Time Handling Concepts

Java Date and Time

Program:

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /*
     * Complete the 'findDay' function below.
     *
     * The function is expected to return a STRING.
     * The function accepts following parameters:
     * 1. INTEGER month
     * 2. INTEGER day
```

* 3. INTEGER year

*/

```
public static String findDay(int month, int day, int year) {
```

```
    Calendar cal = Calendar.getInstance();
```

```
    // Calendar months start from 0, so subtract 1
```

```
    cal.set(year, month - 1, day);
```

```
    return cal.getDisplayName(
```

```
        Calendar.DAY_OF_WEEK,
```

```
        Calendar.LONG,
```

```
        Locale.ENGLISH
```

```
    ).toUpperCase();
```

```
}
```

```
}
```

```
public class Solution {
```

```
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader bufferedReader = new BufferedReader(new  
        InputStreamReader(System.in));
```

```
        BufferedWriter bufferedWriter = new BufferedWriter(new  
        FileWriter(System.getenv("OUTPUT_PATH")));
```

```
        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$",  
        "").split(" ");
```

```
int month = Integer.parseInt(firstMultipleInput[0]);
```

```
int day = Integer.parseInt(firstMultipleInput[1]);
```

```
int year = Integer.parseInt(firstMultipleInput[2]);
```

```
String res = Result.findDay(month, day, year);
```

```
bufferedWriter.write(res);
```

```
bufferedWriter.newLine();
```

```
bufferedReader.close();
```

```
bufferedWriter.close();
```

```
}
```

```
}
```

✓ Test case 0	Compiler Message	
✓ Test case 1	Success	
✓ Test case 2	Input (stdin)	Download
	1 08 05 2015	
✓ Test case 3		
✓ Test case 4	Expected Output	Download
	1 WEDNESDAY	
✓ Test case 5		

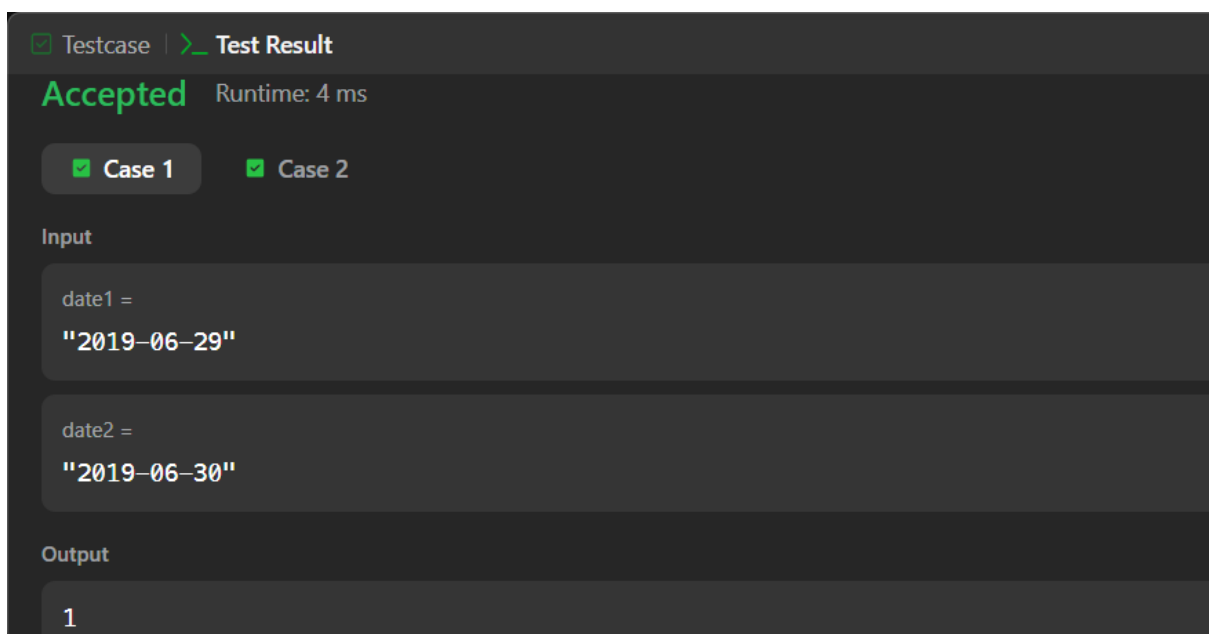
Number of Days Between Two Dates

Program:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

class Solution {
    public int daysBetweenDates(String date1, String date2) {
        LocalDate d1 = LocalDate.parse(date1);
        LocalDate d2 = LocalDate.parse(date2);

        return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
    }
}
```



Session-12: Date and Time Handling Concepts

Day of the Year

Program:

```
class Solution {  
    public int dayOfYear(String date) {  
        int year = Integer.parseInt(date.substring(0, 4));  
        int month = Integer.parseInt(date.substring(5, 7));  
        int day = Integer.parseInt(date.substring(8, 10));  
  
        int[] days = {31,28,31,30,31,30,31,31,30,31,30,31};  
  
        // Check for leap year  
        if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {  
            days[1] = 29;  
        }  
  
        int total = day;  
  
        for (int i = 0; i < month - 1; i++) {  
            total += days[i];  
        }  
  
        return total;  
    }  
}
```

```
}  
}
```

Testcase | Test Result
Accepted Runtime: 0 ms

Case 1 Case 2

Input

date =
"2019-01-09"

Output

9

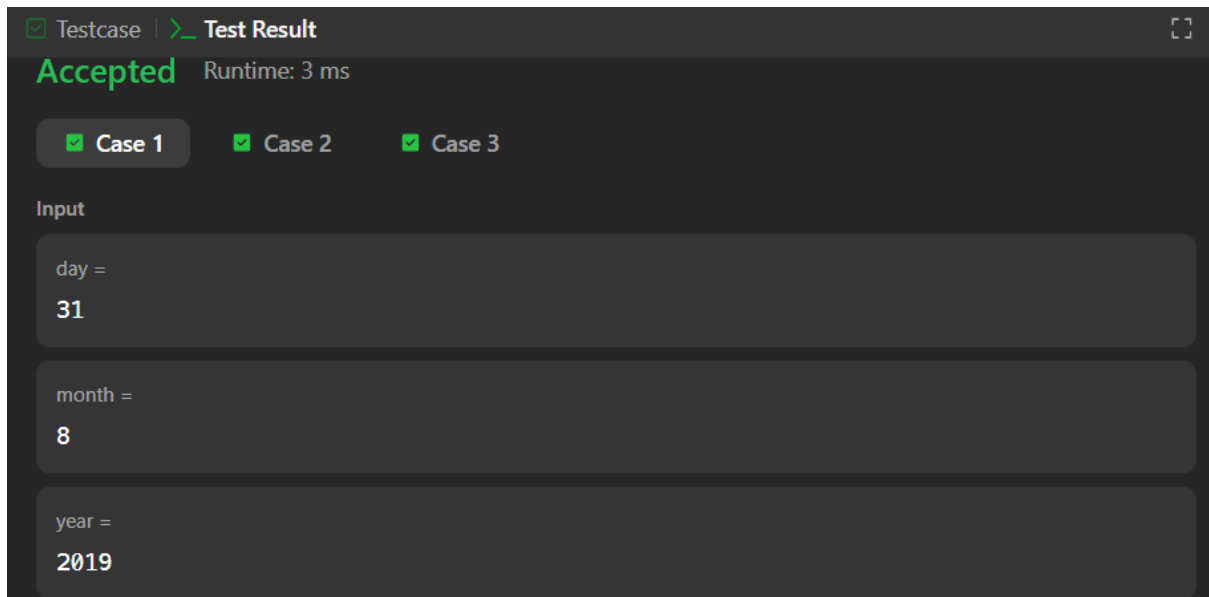
Expected

9

Day of the Week

Program:

```
import java.time.LocalDate;  
  
class Solution {  
    public String dayOfTheWeek(int day, int month, int year) {  
        LocalDate date = LocalDate.of(year, month, day);  
        return date.getDayOfWeek().toString().substring(0, 1)  
            + date.getDayOfWeek().toString().substring(1).toLowerCase();  
    }  
}
```



Session-13: Collection Processing & Transformation Operations

Java Priority Queue

Program:

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.util.PriorityQueue;
import java.util.Comparator;

class Student {
    private final int id;
    private final String name;
    private final double cgpa;

    public Student(int id, String name, double cgpa) {
```

```
    this.id = id;
    this.name = name;
    this.cgpa = cgpa;
}
```

```
public int getId() {
    return id;
}
```

```
public String getName() {
    return name;
}
```

```
public double getCgpa() {
    return cgpa;
}
}
```

```
class Priorities {
    public List<Student> getStudents(List<String> events) {
        // Priority Queue with custom sorting logic
        PriorityQueue<Student> pq = new PriorityQueue<>(
            Comparator.comparing(Student::getCgpa).reversed() // 1. Highest CGPA
first
            .thenComparing(Student::getName) // 2. Alphabetical name if
CGPA ties
            .thenComparing(Student::getId) // 3. Lowest ID if both tie
    }
}
```

```
);
```

```
for (String event : events) {
```

```
    String[] parts = event.split(" ");
```

```
    String command = parts[0];
```

```
    if (command.equals("ENTER")) {
```

```
        String name = parts[1];
```

```
        double cgpa = Double.parseDouble(parts[2]);
```

```
        int id = Integer.parseInt(parts[3]);
```

```
        pq.add(new Student(id, name, cgpa));
```

```
    } else if (command.equals("SERVED")) {
```

```
        pq.poll();
```

```
    }
```

```
}
```

```
// Extract remaining elements in strict priority order
```

```
List<Student> remainingStudents = new ArrayList<>();
```

```
while (!pq.isEmpty()) {
```

```
    remainingStudents.add(pq.poll());
```

```
}
```

```
return remainingStudents;
```

```
}
```

```
}
```

```
public class Solution {  
    private final static Scanner scan = new Scanner(System.in);  
    private final static Priorities priorities = new Priorities();  
  
    public static void main(String[] args) {  
        int totalEvents = Integer.parseInt(scan.nextLine());  
        List<String> events = new ArrayList<>();  
  
        while (totalEvents-- != 0) {  
            String event = scan.nextLine();  
            events.add(event);  
        }  
  
        List<Student> students = priorities.getStudents(events);  
  
        if (students.isEmpty()) {  
            System.out.println("EMPTY");  
        } else {  
            for (Student st: students) {  
                System.out.println(st.getName());  
            }  
        }  
    }  
}
```

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

[Next Challenge](#)

✓ Test case 0

✓ Test case 1 [🔒](#)

✓ Test case 2 [🔒](#)

✓ Test case 3 [🔒](#)

✓ Test case 4 [🔒](#)

✓ Test case 5 [🔒](#)

✓ Test case 6 [🔒](#)

Compiler Message

Success

Input (stdin)

```
1 12
2 ENTER John 3.75 50
3 ENTER Mark 3.8 24
4 ENTER Shafaet 3.7 35
5 SERVED
6 SERVED
7 ENTER Samiha 3.85 36
8 SERVED
9 ENTER Ashley 3.9 42
```

[Download](#)

Java Arraylist

Program:

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);

        // Read the total number of lines
        int n = scan.nextInt();

        // Create a 2D ArrayList to store the lines of numbers
        ArrayList<ArrayList<Integer>> lines = new ArrayList<>();
```

```

for (int i = 0; i < n; i++) {
    int d = scan.nextInt(); // Number of integers on this line
    ArrayList<Integer> row = new ArrayList<>();
    for (int j = 0; j < d; j++) {
        row.add(scan.nextInt());
    }
    lines.add(row);
}

// Read the number of queries
int q = scan.nextInt();

for (int i = 0; i < q; i++) {
    int x = scan.nextInt(); // Line number (1-indexed)
    int y = scan.nextInt(); // Position number (1-indexed)

    // Convert 1-indexed queries to 0-indexed positions
    try {
        System.out.println(lines.get(x - 1).get(y - 1));
    } catch (IndexOutOfBoundsException e) {
        System.out.println("ERROR!");
    }
}

scan.close();
}

```

}

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

[Next Challenge](#)

✓ Test case 0

✓ Test case 1

✓ Test case 2

✓ Test case 3

✓ Test case 4

✓ Test case 5

Compiler Message

Success

Input (stdin)

```
1 5
2 5 41 77 74 22 44
3 1 12
4 4 37 34 36 52
5 0
6 3 20 22 33
7 5
8 1 3
9 3 4
```

[Download](#)

Session-14: Custom Comparators

Largest Number

Program:

```
import java.util.*;

class Solution {

    public String largestNumber(int[] nums) {

        String[] arr = new String[nums.length];

        for (int i = 0; i < nums.length; i++) {

            arr[i] = String.valueOf(nums[i]);

        }

        Arrays.sort(arr, (a, b) -> {

            String ab = a + b;
```

```

        String ba = b + a;

        return ba.compareTo(ab);
    });
    if (arr[0].equals("0")) {
        return "0";
    }

    StringBuilder result = new StringBuilder();

    for (String s : arr) {
        result.append(s);
    }

    return result.toString();
}
}

```

☒ Testcase
 ☒ **Test Result**

Accepted
Runtime: 1 ms

☒ **Case 1**
☒ **Case 2**

Input

nums =
 [10,2]

Output

"210"

Expected

"210"

Session-15: Custom Comparators & Sorting Logic

Sort Characters by Frequency

Program:

```
import java.util.*;

class Solution {

    public String frequencySort(String s) {

        Map<Character, Integer> freq = new HashMap<>();
        for (char c : s.toCharArray()) {
            freq.put(c, freq.getDefault(c, 0) + 1);
        }

        PriorityQueue<Character> maxHeap = new PriorityQueue<>(
            (a, b) -> freq.get(b) - freq.get(a)
        );

        maxHeap.addAll(freq.keySet());

        StringBuilder result = new StringBuilder();

        while (!maxHeap.isEmpty()) {

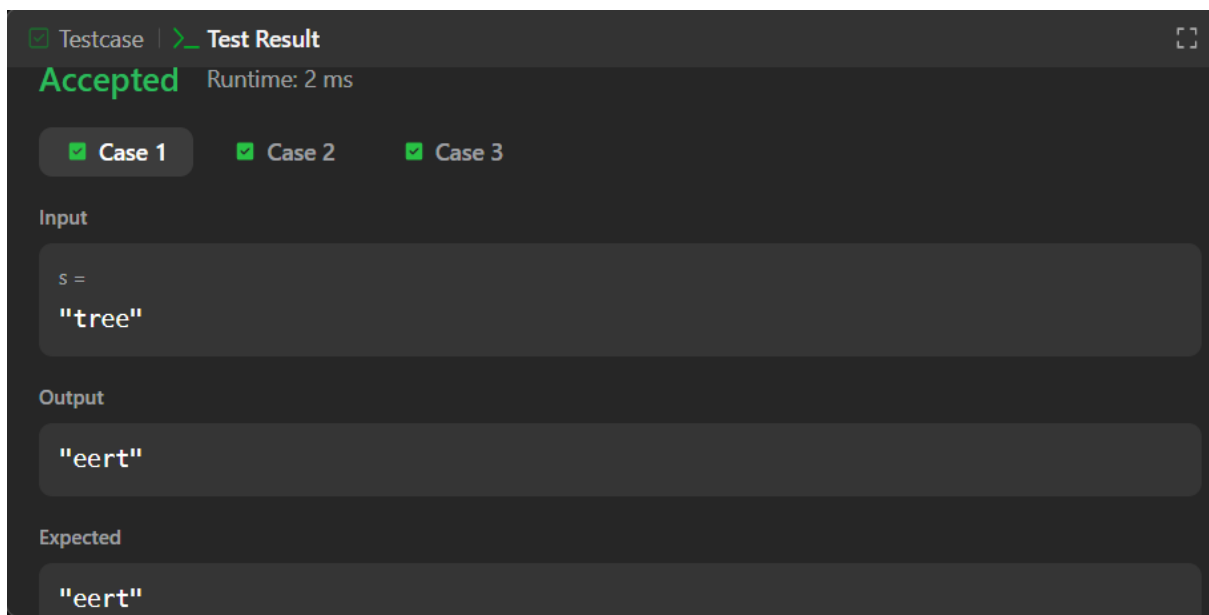
            char c = maxHeap.poll();
            int count = freq.get(c);

            for (int i = 0; i < count; i++) {
```

```

        result.append(c);
    }
}
return result.toString();
}
}

```



Java Sort

Program:

```

import java.util.*;

class Student{
    private int id;
    private String fname;
    private double cgpa;
    public Student(int id, String fname, double cgpa) {

```

```

        super();
        this.id = id;
        this.fname = fname;
        this.cgpa = cgpa;
    }
    public int getId() {
        return id;
    }
    public String getFname() {
        return fname;
    }
    public double getCgpa() {
        return cgpa;
    }
}

```

//Complete the code

```

public class Solution
{
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());

        List<Student> studentList = new ArrayList<Student>();
        while(testCases>0){
            int id = in.nextInt();

```

```

        String fname = in.next();
        double cgpa = in.nextDouble();

        Student st = new Student(id, fname, cgpa);
        studentList.add(st);

        testCases--;
    }
    Collections.sort(studentList, new Comparator<Student>() {
@Override
public int compare(Student a, Student b) {
    if (a.getCgpa() != b.getCgpa()) {
        return Double.compare(b.getCgpa(), a.getCgpa());
    }

    int nameCompare = a.getFname().compareTo(b.getFname());

    if (nameCompare != 0) {
        return nameCompare;
    }
    return Integer.compare(a.getId(), b.getId());
}
});

for(Student st: studentList){
    System.out.println(st.getFname());
}

```

```
}  
  
}
```

✔ Test case 0

✔ Test case 1

✔ Test case 2

✔ Test case 3

✔ Test case 4

✔ Test case 5

✔ Test case 6

Compiler Message

Success

Input (stdin)

1

5

2

33 Rumpa 3.68

3

85 Ashis 3.85

4

56 Samiha 3.75

5

19 Samara 3.75

6

22 Fahim 3.76

Expected Output

1

Ashis

Download

Download